

Information Technology University, Lahore

Applied Machine Learning

BSCE-23 | Spring-2026

Assignment # 04	Total Marks = 100
Issue Date: 5/11 / 2026	Due Date: 5 / 17 / 2026

Instructions

- Please review the University Plagiarism Policy.
- Any submissions handed in late will not be accepted and will receive a zero score.
- Assignment should be uploaded as PDF file with source file in Google Classroom.
- The name of the file should be your Roll Number as: BSCEXXXX.
- Please submit your own work only.
- You must submit a .py source code file along with your report.
- There should be a Report.pdf detailing your experience and results (do NOT explain code — explain results only).

PART A | Convolutional Neural Network (CNN)

Marks: 50

Overview

Develop a custom Convolutional Neural Network (CNN) to classify plant disease images into multiple categories. The goal is to build, train, and evaluate a CNN model from scratch using the PlantVillage Dataset. This task simulates a real-world agricultural AI application where accurate disease detection can significantly impact crop yields.

Dataset: PlantVillage Dataset

Why PlantVillage? This dataset is ideal for a multi-class CNN image classification task. It contains 54,000+ high-quality RGB images across 38 disease and healthy plant categories, making it rich enough to demonstrate CNN depth and regularization techniques.

Dataset Name	PlantVillage Disease Classification Dataset
No. of Classes	38 (healthy + diseased categories across 14 crop types)
Total Images	~54,306 color images (256x256 px)
Format	JPEG images, folder-structured by class label
Available At	https://www.kaggle.com/datasets/emmarex/plantdisease
Alternate Source	https://github.com/spMohanty/PlantVillage-Dataset
License	Creative Commons Attribution 4.0 (CC BY 4.0)

Tasks

Task 1: Load and Explore the Dataset [10 Marks]

- Download the PlantVillage dataset and load it into your Python environment using os and PIL/OpenCV.
- Load images and labels manually — do NOT use built-in dataset loaders or ImageDataGenerator.
- Normalize pixel values to the range [0, 1].
- Manually split the dataset into 80% training and 20% validation:
 - Do NOT use ImageDataGenerator.split() or train_test_split().
 - Save splits into two separate folders: train/ and val/ programmatically.
- Visualize at least 2 sample images from each of 5 different classes with labels displayed.
- Print class distribution (number of images per class) as a bar chart.

Task 2: Build a Custom CNN from Scratch [15 Marks]

- Build your CNN entirely from scratch — no pre-trained models (VGG, ResNet, MobileNet, etc.).
- Your CNN architecture must include:
 - At least 5 Convolutional layers with ReLU activation.
 - BatchNormalization after each convolutional layer.
 - Dropout (rate ≥ 0.3) after at least 3 convolutional layers.
 - MaxPooling after each convolutional block.
 - A Global Average Pooling or Flatten layer before the dense layers.
 - At least 2 fully connected (Dense) layers.
 - A Softmax output layer for multi-class classification.
- Print the complete model summary showing all layers, output shapes, and parameter counts.

Task 3: Train the Model [10 Marks]

- Compile the model using the Adam optimizer and categorical cross-entropy loss.
- Train the model for at least 25 epochs using the training split.
- Use the validation split to monitor training performance.
- Apply at least one Learning Rate Scheduler (e.g., ReduceLROnPlateau or CosineDecay).
- Plot training and validation accuracy/loss curves over epochs.

Task 4: Evaluate the Model [10 Marks]

- Evaluate your model on the validation set and report the following:
 - Overall Accuracy
 - Per-class Precision, Recall, and F1-Score
 - Confusion Matrix (write a custom method to compute TP, TN, FP, FN for each class)
- Visualize the Confusion Matrix as a heatmap using Matplotlib or Seaborn.
- Identify the top 3 most confused class pairs and briefly explain why.

Task 5: Report Your Findings [5 Marks]

- Summarize your CNN architecture decisions and why you made them.
- Include training/validation accuracy and loss curves in your report.
- Discuss challenges faced (e.g., class imbalance, overfitting) and improvements tried.
- Comment on model performance — which classes were hardest to classify and why?

PART B | Recurrent Neural Network (RNN / LSTM)

Marks: 50

Overview

Build a Recurrent Neural Network (RNN) using LSTM layers to perform sentiment analysis on movie reviews. The model should classify each review as positive or negative. This task demonstrates the power of sequential models in understanding context and temporal dependencies in natural language.

Dataset: IMDb Movie Reviews Dataset

Why IMDb Reviews? This is a classic NLP benchmark dataset with 50,000 labeled movie reviews (25K train / 25K test). It is perfectly suited for binary sentiment classification using LSTM/RNN architectures. The dataset is large enough to demonstrate sequence modeling challenges like vanishing gradients and the benefit of gating mechanisms.

Dataset Name	IMDb Large Movie Review Dataset (Sentiment Analysis)
No. of Samples	50,000 reviews (25,000 train / 25,000 test)
Classes	Binary: Positive / Negative sentiment
Avg. Review Length	~231 words per review
Format	Raw text files, folder-structured by sentiment
Available At	https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
Alternate Source	https://ai.stanford.edu/~amaas/data/sentiment/
License	For academic/research use

Tasks

Task 1: Load and Preprocess the Dataset [10 Marks]

- Download and load the IMDb dataset from raw text files (do NOT use Keras built-in `imdb.load_data()`).
- Perform the following preprocessing manually:
 - Convert all text to lowercase.
 - Remove HTML tags, punctuation, and stop words.
 - Build a vocabulary from the training data (limit to top 20,000 most frequent words).
 - Convert each review to a sequence of integer token IDs.
 - Pad/truncate all sequences to a fixed length of 300 tokens.
- Split the dataset: 80% training, 20% validation (manual split — no `train_test_split()`).
- Visualize the distribution of review lengths before and after padding.

Task 2: Build a Custom LSTM-based RNN Model [15 Marks]

- Build your RNN from scratch using LSTM or GRU layers — no pre-trained language models (BERT, GPT, etc.).
- Your RNN architecture must include:
 - An Embedding layer (trainable, initialized randomly — no pre-trained GloVe/Word2Vec).
 - At least 2 stacked LSTM or GRU layers with `return_sequences=True` on intermediate layers.
 - Dropout and Recurrent Dropout in LSTM layers.
 - BatchNormalization between LSTM layers.
 - At least 1 Dense layer after the recurrent layers.
 - A Sigmoid output neuron for binary classification.
- Print the model summary with all layer names, output shapes, and parameter counts.

Task 3: Train the Model [10 Marks]

- Compile the model using the Adam optimizer and binary cross-entropy loss.
- Train for at least 15 epochs using mini-batches of size 64.
- Implement Early Stopping (`patience=3`) to prevent overfitting.
- Log and plot training and validation accuracy/loss curves.
- Report the best validation accuracy achieved and at which epoch.

Task 4: Evaluate the Model [10 Marks]

- Evaluate the trained model on the validation set and report:
 - Accuracy, Precision, Recall, and F1-Score.
 - ROC Curve and AUC Score (plot using Matplotlib).
 - Confusion Matrix (write a custom function — do not use sklearn's `confusion_matrix`).
- Test your model on at least 5 custom reviews that you write yourself — display the model's predictions.
- Visualize the top 20 most frequent words in positive vs. negative reviews (use word frequency bar charts).

Task 5: Report Your Findings [5 Marks]

- Explain your LSTM architecture choices (number of layers, hidden units, embedding size).
- Discuss how the model handles long-range dependencies in text.
- Describe any overfitting observed and how you addressed it.
- Compare LSTM performance against a simple baseline (e.g., majority class predictor or word-count model).

#	Task Description	Part	Marks
1	Load & Explore Dataset (Images)	Part A — CNN	10
2	Build Custom CNN Architecture	Part A — CNN	15
3	Train CNN Model	Part A — CNN	10
4	Evaluate CNN Model	Part A — CNN	10
5	Report Findings (CNN)	Part A — CNN	5
6	Load & Preprocess Dataset (Text)	Part B — RNN	10
7	Build Custom LSTM Model	Part B — RNN	15
8	Train RNN Model	Part B — RNN	10
9	Evaluate RNN Model	Part B — RNN	10
10	Report Findings (RNN)	Part B — RNN	5
TOTAL			100

Submission Requirements

File 1	Python source code: .py file (named as your Roll Number, e.g., BSCE0001.py)
File 2	Report: Report.pdf — results, analysis, curves, confusion matrices
Upload To	Google Classroom (PDF + .py code file only)
File Naming	Roll Number only — e.g., BSCE0001_A04.py and BSCE0001_A04_Report.pdf
Late Policy	Zero marks for late submissions — no exceptions
Plagiarism	University Plagiarism Policy strictly enforced — zero for all parties involved

Important Note: Your report should NOT explain your code. It should explain your results, observations, challenges, and learnings. Plots, confusion matrices, and accuracy curves must be included in the report.